COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 031	**SECTION:**	CPE21S4
DATE PERFORMED:	24/07/2026	**DATE SUBMITTED:**	29/07/2026
NAME:	JAMES DANIEL M. VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 3.1 **Linked Lists**

1. PROCEDURE OUTPUT

Screenshot

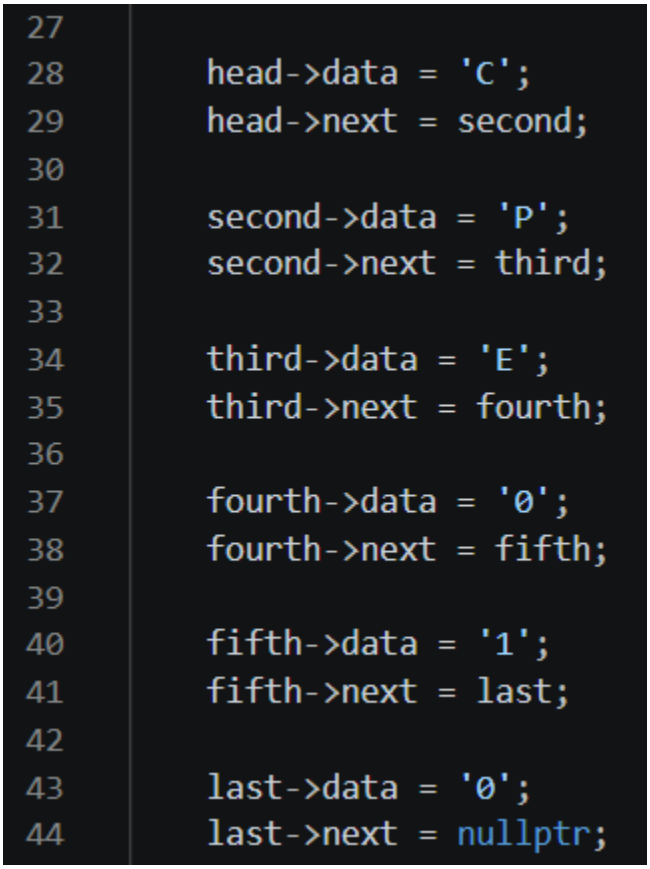
```
#include <iostream>
#include <utility>
// #include "doubly_ll.h"

class Node {
public:
    char data;
    Node *next;
};

int main()
{

    Node *head = NULL;
    Node *second = NULL;
    Node *third = NULL;
    Node *fourth = NULL;
    Node *fifth = NULL;
    Node *last = NULL;

    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
    fifth = new Node;
    last = new Node;
```



```
27
28      head->data = 'C';
29      head->next = second;
30
31      second->data = 'P';
32      second->next = third;
33
34      third->data = 'E';
35      third->next = fourth;
36
37      fourth->data = '0';
38      fourth->next = fifth;
39
40      fifth->data = '1';
41      fifth->next = last;
42
43      last->data = '0';
44      last->next = nullptr;
```

Discussion	

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**

Traversal

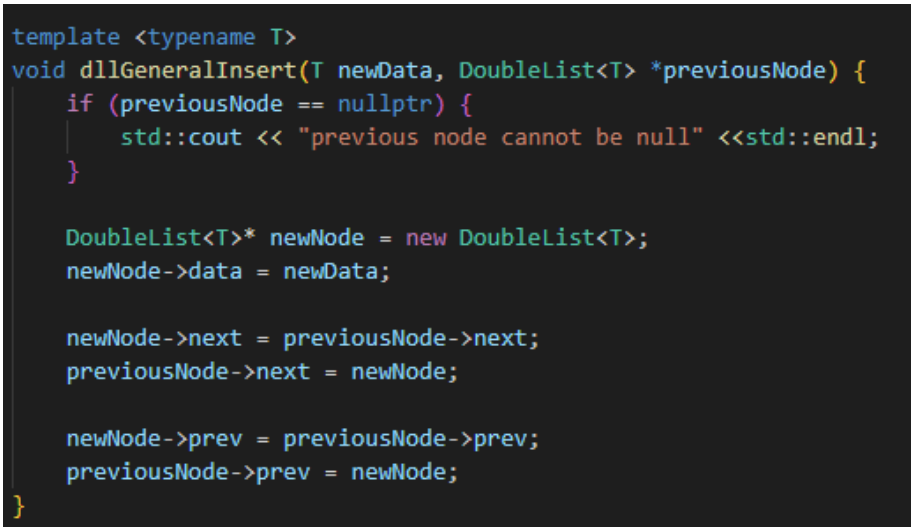
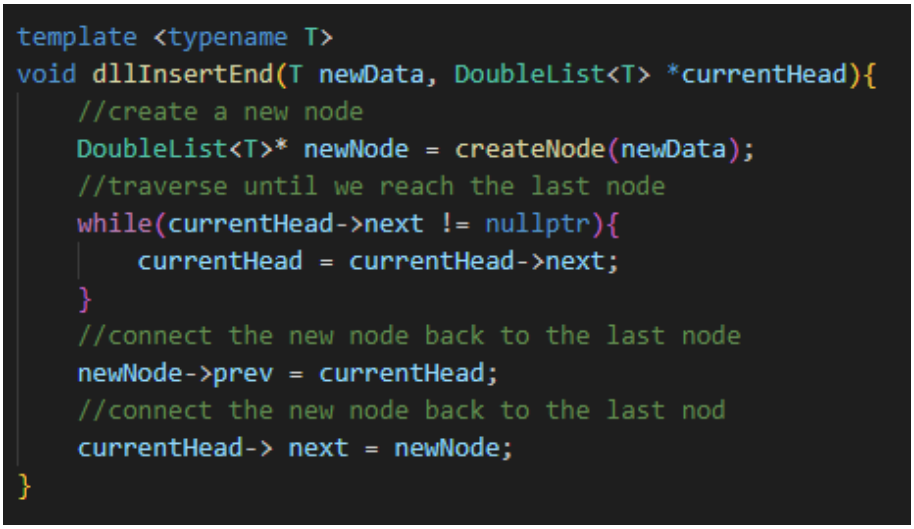
```
template <typename T>
void dllTraverse(DoubleList<T>* currentNode){
    //check if the list is empty
    if(currentNode == nullptr) {
        std::cout << "The list is empty";
        return;
    }

    DoubleList<T>* tail;
    std::cout << "FORWARD" << std::endl;
    //forward traverse
    while (currentNode != nullptr){
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << std::endl << "REVERSE" << std::endl;
    //reverse traverse
    while(tail != nullptr){
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << std::endl;
};
```

Insertion at head

```
//inserting a node to the head
template <typename T>
void dllInsertHead(T newData, DoubleList<T>** currentNode){

    //create an new node:
    DoubleList<T> *newNode = createNode(newData);
    //point the newNode to the currentNode (head)
    newNode->next = *currentNode;
    //point the currentNode(head) to the newNode;
    (*currentNode)->prev = newNode;
    //updating the header pointer
    *currentNode = newNode;
};
```

Insertion at any part of the list	
Insert at the end	

```
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T> *previousNode) {
    if (previousNode == nullptr) {
        std::cout << "previous node cannot be null" <<std::endl;
    }

    DoubleList<T>* newNode = new DoubleList<T>;
    newNode->data = newData;

    newNode->next = previousNode->next;
    previousNode->next = newNode;

    newNode->prev = previousNode->prev;
    previousNode->prev = newNode;
}
```

```
template <typename T>
void dllInsertEnd(T newData, DoubleList<T> *currentHead){
    //create a new node
    DoubleList<T>* newNode = createNode(newData);
    //traverse until we reach the last node
    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }
    //connect the new node back to the last node
    newNode->prev = currentHead;
    //connect the new node back to the last nod
    currentHead-> next = newNode;
}
```

Deletion at the end	

```cpp
template <typename T>
void dllDelNode(T findData, DoublyList<T> **head)
{
    if(*head == nullptr) return;

    //Traverse to the tail of the list
    DoublyList<T>* currNode =  *head;
    while (currNode->next != nullptr) {
        currNode = currNode->next;
    }

    // Traverse backward from tail to head to find the data
    while(currNode != nullptr && currNode->data != findData){
        currNode = currNode->prev;
    }

    // if the data wasn't found in the list
    if(currNode == nullptr) return;
    // Update the 'next' pointer of the previous node
    if(currNode->prev != nullptr){
        currNode->prev->next = currNode->next;
    }
    else {
        *head = currNode->next;
    }
    // Update the 'prev' pointer of the next node
    if (currNode->next != nullptr) {
        currNode->next->prev = currNode->prev;
    }

    delete currNode;

}
```

Table 3-2. Code for the List Operations

a.	Source Code	

```cpp
template <typename T>
void dllTraverse(DoublyList<T> *currentNode)
{
    // check if the list is empty
    if (currentNode == nullptr)
    {
        std::cout << "The list is empty:" <<
std::endl;
        return;
```

```
    }
    DoublyList<T> *tail;
    std::cout << "FORWARD" << std::endl;
    // forward traverse
    while (currentNode != nullptr)
    {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
```

Console

```
(a) Traverse the list.
FORWARD
C P E 1 0 1
```

b.

Source Code

```
template <typename T>
void dllInsertHead(T newData, DoublyList<T>
**currentNode)
{
    // create a new node
    DoublyList<T> *newNode =
createNode(newData);

    // print the newNode to the currentNode
(head)
    newNode->next = *currentNode;
    // print the currentNode (head) to the
newNode
    (*currentNode)->prev = newNode;
    // update the header pointer
    *currentNode = newNode;
}
```

Console

```
(b) Insert the element 'G'.
FORWARD
G C P E 1 0 1
```

c.

Source Code

```
template <typename T>
void dllInsertGen(T newData, DoublyList<T>
*currentNode) {

    DoublyList<T>* newNode =
createNode(newData);

    newNode->next = currentNode->next;
    newNode->prev = currentNode;

    if(currentNode->next != nullptr) {
        currentNode->next->prev = newNode;
    }

    currentNode->next = newNode;
}
```

Console

```
(c) insert an element 'E' with the previous node element being 'P'.
FORWARD
G C P E E 1 0 1
```

d.

Source Code

```
template <typename T>
void dllDelNode(T findData, DoublyList<T>
**head)
{

    if(*head == nullptr) return;

    DoublyList<T>* currNode = *head;
    DoublyList<T>* prevNode = nullptr;

    while(currNode != nullptr && currNode->data
!= findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }

    if(currNode == nullptr) return;
```

```
    if(prevNode == nullptr){
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Console

```
(d) Delete the node containing the element 'C'.
FORWARD
G P E E 1 0 1
```

e.

Source Code

```
template <typename T>
void dllDelNode(T findData, DoublyList<T>
**head)
{

    if(*head == nullptr) return;

    DoublyList<T>* currNode = *head;
    DoublyList<T>* prevNode = nullptr;

    while(currNode != nullptr && currNode->data
!= findData) {
        prevNode = currNode;
        currNode = currNode->next;
    }

    if(currNode == nullptr) return;

    if(prevNode == nullptr){
        *head = currNode->next;
    } else {
        prevNode->next = currNode->next;
    }
```

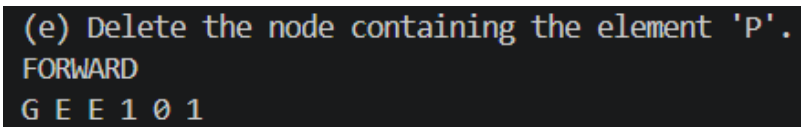
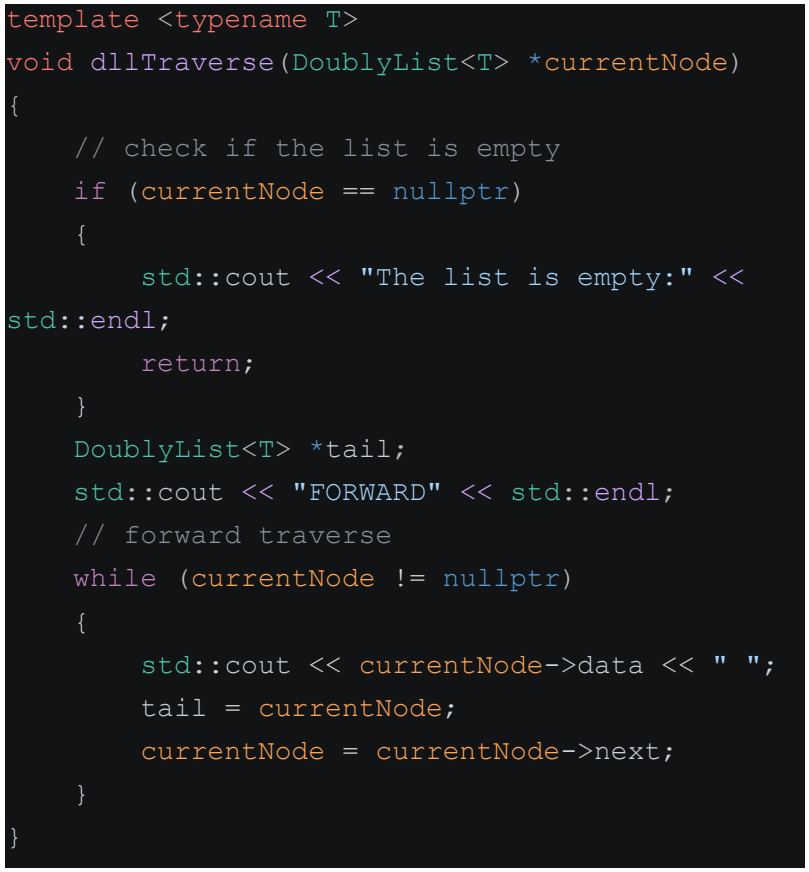
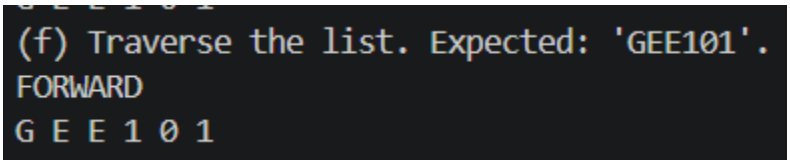
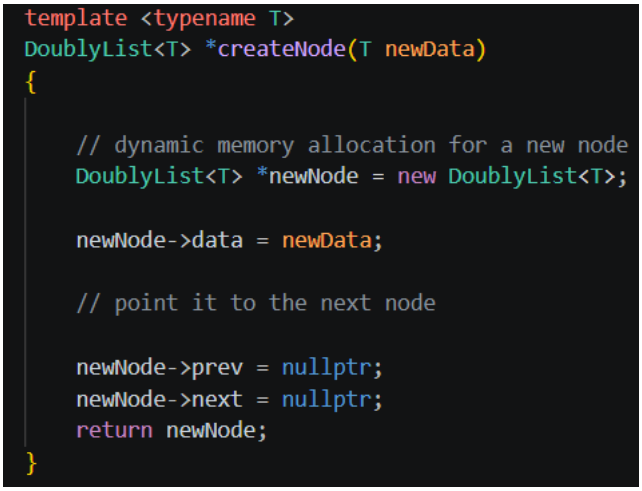
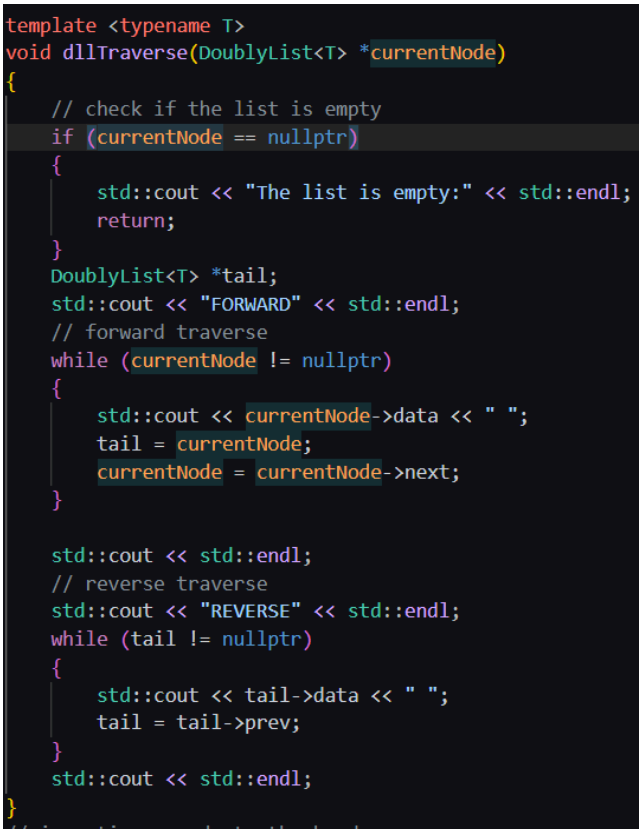
		
	Console	
f.	Source Code	
	Console	

Table 3-3. Code and Analysis for Doubly Linked Lists

Screenshot(s)	Analysis


``
template <typename T>
DoublyList<T> *createNode(T newData)
{
 	
    // dynamic memory allocation for a new node
    DoublyList<T> *newNode = new DoublyList<T>;
 	
    newNode->data = newData;
 	
    // point it to the next node
 	
    newNode->prev = nullptr;
    newNode->next = nullptr;
    return newNode;
}
``
	Allocates memory dynamically for **DoublyList<T>**. Unlike the singly linked list that was discussed, it initializes **two pointers (next and prev)** to **nullptr** alongside the data, ensuring the node is safely isolated before linking into one another to create the list. The function's parameter only inquires the data for the node to initialize.


``
template <typename T>
void dllTraverse(DoublyList<T> *currentNode)
{
    // check if the list is empty
    if (currentNode == nullptr)
    {
        std::cout << "The list is empty:" << std::endl;
        return;
    }
    DoublyList<T> *tail;
    std::cout << "FORWARD" << std::endl;
    // forward traverse
    while (currentNode != nullptr)
    {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
 	
    std::cout << std::endl;
    // reverse traverse
    std::cout << "REVERSE" << std::endl;
    while (tail != nullptr)
    {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << std::endl;
}
``
	This function **iterates** forward through next links to print the data while capturing the **tail node**, then loops backward using **prev links** to print the sequence in **reverse order**. The **tail node** is how it reads the reverse order of the traversal list. Once it reaches the nullptr of the end, thus, it reads the tail's node to reverse traverse the list. As represented as:
	
	tail = tail->prev;
	
	This representation is the initialization of tail node:
	
	tail = currentNode;
	currentNode = currentNode->next;

```
template <typename T>
void dllInsertHead(T newData, DoublyList<T> **currentNode)
{
    // create a new node
    DoublyList<T> *newNode = createNode(newData);

    // print the newNode to the currentNode (head)
    newNode->next = *currentNode;
    // print the currentNode (head) to the newNode
    (*currentNode)->prev = newNode;
    // update the header pointer
    *currentNode = newNode;
}
```

Modifies the head update function. Beyond pointing **newNode->next** to the **old head**, this represents that, it sets **(*head)->prev** = **newNode** to establish the backward link before reassigning the head pointer to the new node.

```
template <typename T>
void dllInsertGen(T newData, DoublyList<T> *currentNode) {

    DoublyList<T>* newNode = createNode(newData);

    newNode->next = currentNode->next;
    newNode->prev = currentNode;

    if(currentNode->next != nullptr) {
        currentNode->next->prev = newNode;
    }

    currentNode->next = newNode;
}
```

The **dllInsertGen function** is a function that inserts an item after a **target node**. It updates four pointers instead of two:

newNode->next;
newNode->prev;

the target node's original successor's prev link, represented as:

currentNode->next->prev = newNode;

and finally the target node's next link:

currentNode->next = newNode;

The if structure is where you are inquiring and equating the **currentNode->next->prev**, will be initialized into the newly created object named **newNode.**

```
template <typename T>
void dllDelNode(T findData, DoublyList<T> **head)
{

    if(*head == nullptr) return;

    //Traverse to the tail of the list
    DoublyList<T>* currNode =  *head;
    while (currNode->next != nullptr) {
        currNode = currNode->next;
    }

    // Traverse backward from tail to head to find the data
    while(currNode != nullptr && currNode->data != findData)
        currNode = currNode->prev;
    }

    // if the data wasn't found in the list
    if(currNode == nullptr) return;
    // Update the 'next' pointer of the previous node
    if(currNode->prev != nullptr){
        currNode->prev->next = currNode->next;
    }
    else {
        *head = currNode->next;
    }
    // Update the 'prev' pointer of the next node
    if (currNode->next != nullptr) {
        currNode->next->prev = currNode->prev;
    }

    delete currNode;

}
```

The **dllDelNode function** Unlinks a node without a trailing pointer. It reads **currNode->prev** and **currNode->next** to bridge its sides nodes directly **(prev->next = next and next->prev = prev)**, safely bypassing and delete-ing the node. This function safely handles tricky situations like empty lists, single-item lists, and removing the first or last items without leaking memory.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

[SOURCE CODES]

```
#include <iostream>
#include <string>
#include <chrono>
#include <thread>
#include "playlist_ll.h"

// function to print the interactive options menu
void printMenu() {
    std::cout << "\n================ MUSIC PLAYER MENU ================"
<< std::endl;
    std::cout << " 1. Next Song" << std::endl;
```

```
    std::cout << " 2. Previous Song" << std::endl;
    std::cout << " 3. Add Song to Beginning (Head)" << std::endl;
    std::cout << " 4. Remove Current Song" << std::endl;
    std::cout << " 5. Play/Display All Songs Once" << std::endl;
    std::cout << " 6. Exit Player" << std::endl;
    std::cout << "==================================================" <<
std::endl;
    std::cout << "Enter your choice (1-6): ";
}

int main()
{
    // Initialize an empty playlist
    DoublyList<std::string> *head = nullptr;

    // Track the currently playing song node pointer
    DoublyList<std::string> *currentPlaying = nullptr;

    int choice = 0;
    std::string songName;

    std::cout << "Welcome to the Circular Playlist Creator!" <<
std::endl;

    while (choice != 6) {
        // Display current playing status
        std::cout <<
"\n--------------------------------------------------" << std::endl;
        if (currentPlaying != nullptr) {
            std::cout << "NOW PLAYING: [ " << currentPlaying->data << "
]" << std::endl;
        } else {
            std::cout << "NOW PLAYING: [ No songs in playlist ]" <<
std::endl;
        }

        printMenu();
        std::cin >> choice;
```

```
        // Error checking
        if (std::cin.fail()) {
            std::cin.clear();
            std::cin.ignore(10000, '\n');
            std::cout << "Invalid input. Please enter a valid number
(1-6)." << std::endl;
            continue;
        }

        // Clear input buffer before reading dynamic string names
        std::cin.ignore();

        switch (choice) {
            case 1: // Next Song
                if (currentPlaying != nullptr) {
                    currentPlaying = currentPlaying->next;
                } else {
                    std::cout << "The playlist is empty." << std::endl;
                }
                break;

            case 2: // Previous Song
                if (currentPlaying != nullptr) {
                    currentPlaying = currentPlaying->prev;
                } else {
                    std::cout << "The playlist is empty." << std::endl;
                }
                break;

            case 3: // Add Song to Head
                std::cout << "Enter song name to add at beginning: ";
                std::getline(std::cin, songName);
                dllInsertHead<std::string>(songName, &head);

                if (currentPlaying == nullptr) {
                    currentPlaying = head;
                }
```

```
                std::cout << "Added '" << songName << "' to the
beginning." << std::endl;
                break;

            case 4: // Remove Current Song
                if (currentPlaying == nullptr) {
                    std::cout << "Playlist is already empty." <<
std::endl;
                } else {
                    DoublyList<std::string> *nodeToDelete =
currentPlaying;

                    // Move currentPlaying pointer to the next track
before deleting the target node
                    if (currentPlaying->next == currentPlaying) {
                        currentPlaying = nullptr;
                    } else {
                        currentPlaying = currentPlaying->next;
                    }

                    std::cout << "Removing song: '" << nodeToDelete->data
<< "'" << std::endl;
                    dllDelNode<std::string>(&head, nodeToDelete);
                }
                break;

            case 5: // Play/Display All Songs Once
                std::cout << "\n--- Full Playlist Status ---" <<
std::endl;
                dllTraverse(head);
                break;

            case 6: // Exit Player
                std::cout << "Thank you for using the Music Player.
Goodbye!" << std::endl;
                break;

            default:
```

```
                std::cout << "Invalid choice selection. Try numbers 1
through 8." << std::endl;
                break;
        }
    }

    return 0;
}
```

[HEADER FILE CODES]

```
#ifndef PLAYLIST_LL_H
#define PLAYLIST_LL_H

#include <iostream>
#include <string>

// Reusing your exact DoublyList node structure
template <typename T>
class DoublyList {
public:
    T data;
    DoublyList<T> *next;
    DoublyList<T> *prev;
};

// Create a node function
template <typename T>
DoublyList<T> *createNode(T newData) {
    DoublyList<T> *newNode = new DoublyList<T>;
    newNode->data = newData;

    // In a single-node circular list, it points to itself initially
    newNode->prev = newNode;
    newNode->next = newNode;
    return newNode;
}
```

```cpp
// Traverse the playlist (Prints all songs exactly once per the activity guideline)
template <typename T>

void dllTraverse(DoublyList<T> *head) {
    if (head == nullptr)
    {
        std::cout << "The playlist is empty." << std::endl;
        return;
    }

    DoublyList<T> *currentNode = head;
    std::cout << "\n>>> PLAYING ENTIRE PLAYLIST (LOOP ONCE) <<<" << std::endl;

    do {
        // Print the active song
        std::cout << " -> Playing: [ " << currentNode->data << " ]" << std::endl;

        // Pause the program for 1 second before scrolling to the next node
        std::this_thread::sleep_for(std::chrono::milliseconds(1000));

        currentNode = currentNode->next;
    } while (currentNode != head); // Loops through the structure until it hits the starting head

    std::cout << "✔ Reached the end of the playlist loop.\n" << std::endl;
}

// Inserting a node at the head
template <typename T>
void dllInsertHead(T newData, DoublyList<T> **head) {
    DoublyList<T> *newNode = createNode(newData);

    if (*head == nullptr) {
```

```
        *head = newNode;
        return;
    }

    DoublyList<T> *tail = (*head)->prev;

    newNode->next = *head;
    newNode->prev = tail;

    tail->next = newNode;
    (*head)->prev = newNode;

    *head = newNode;
}

// General Insertion
template <typename T>
void dllInsertGen(DoublyList<T> *targetNode, T newData) {
    if (targetNode == nullptr) return;

    DoublyList<T> *newNode = createNode(newData);

    newNode->next = targetNode->next;
    newNode->prev = targetNode;

    targetNode->next->prev = newNode;
    targetNode->next = newNode;
}

// Delete Node
template <typename T>
void dllDelNode(DoublyList<T> **head, DoublyList<T> *delNode) {
    if (*head == nullptr || delNode == nullptr) return;

    // Case 1: Only one node in the list
    if (delNode->next == delNode && delNode->prev == delNode) {
        *head = nullptr;
    }
```

```
    else {
        delNode->prev->next = delNode->next;
        delNode->next->prev = delNode->prev;

        // Case 2: If we are deleting the active head pointer, move head
forward
        if (*head == delNode) {
            *head = delNode->next;
        }
    }

    delete delNode;
}

#endif
```

[Outputs]

>>> printMenu() function <<<

```
------------------------------------------------
NOW PLAYING: [ No songs in playlist ]

================ MUSIC PLAYER MENU ================
 1. Next Song
 2. Previous Song
 3. Add Song to Beginning
 4. Remove Current Song
 5. Play/Display All Songs Once
 6. Exit Player
================================================
Enter your choice (1-6):
```

>>> dllInsertHead() function (Add song) <<<

```
--------------------------------------------------
NOW PLAYING: [ No songs in playlist ]

================ MUSIC PLAYER MENU ================
 1. Next Song
 2. Previous Song
 3. Add Song to Beginning
 4. Remove Current Song
 5. Play/Display All Songs Once
 6. Exit Player
==================================================
Enter your choice (1-6): 3
Enter song name to add: BUWAN
```

```
--------------------------------------------------
NOW PLAYING: [ BUWAN ]

================ MUSIC PLAYER MENU ================
 1. Next Song
 2. Previous Song
 3. Add Song to Beginning
 4. Remove Current Song
 5. Play/Display All Songs Once
 6. Exit Player
==================================================
Enter your choice (1-6):
```

>>> next case function (next song) <<<

```
NOW PLAYING: [ BUWAN ]

================ MUSIC PLAYER MENU ================
 1. Next Song
 2. Previous Song
 3. Add Song to Beginning
 4. Remove Current Song
 5. Play/Display All Songs Once
 6. Exit Player
===================================================
Enter your choice (1-6): 1

---------------------------------------------------
NOW PLAYING: [ BULONG ]

================ MUSIC PLAYER MENU ================
 1. Next Song
 2. Previous Song
 3. Add Song to Beginning
 4. Remove Current Song
 5. Play/Display All Songs Once
 6. Exit Player
===================================================
Enter your choice (1-6):
```

>>> Previous Case function (next song) <<<

```
--------------------------------------------------
NOW PLAYING: [ BULONG ]

================ MUSIC PLAYER MENU ================
 1. Next Song
 2. Previous Song
 3. Add Song to Beginning
 4. Remove Current Song
 5. Play/Display All Songs Once
 6. Exit Player
==================================================
Enter your choice (1-6): 2

--------------------------------------------------
NOW PLAYING: [ BUWAN ]

================ MUSIC PLAYER MENU ================
 1. Next Song
 2. Previous Song
 3. Add Song to Beginning
 4. Remove Current Song
 5. Play/Display All Songs Once
 6. Exit Player
==================================================
Enter your choice (1-6):
```

>>> dllDelNode() function (remove current song) <<<

```
================================================
Enter your choice (1-6): 4
Removing song: 'BUWAN'

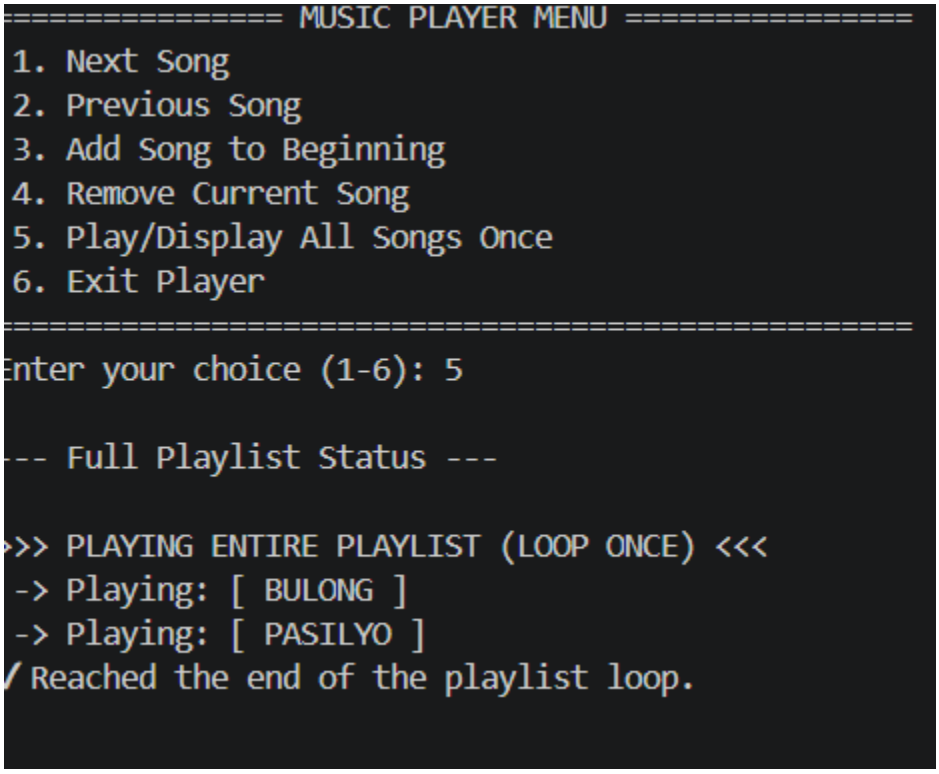
------------------------------------------------
NOW PLAYING: [ BULONG ]

=============== MUSIC PLAYER MENU ===============
 1. Next Song
 2. Previous Song
 3. Add Song to Beginning
 4. Remove Current Song
 5. Play/Display All Songs Once
 6. Exit Player
================================================
Enter your choice (1-6): 5

--- Full Playlist Status ---

>>> PLAYING ENTIRE PLAYLIST (LOOP ONCE) <<<
 -> Playing: [ BULONG ]
 -> Playing: [ PASILYO ]
✓ Reached the end of the playlist loop.
```

>>> dllTraverse() function (display all songs) <<<



CODE ANALYSIS:

In the supplementary activity, we are tasked to create a C++ program implementing the topic discussed, the linked list. The program we are tasked to create is a **music playlist**. In creating the playlist, the activity states that we should have functions supporting:

- **Function 1:** *(Create a playlist using multiple songs.)*
- **Function 2:** *(Add songs to the playlist.)*
- **Function 3:** *(Remove a song from the playlist.)*
- **Function 4:** *(Play songs in a loop.)*

With this function, we will be able to finally call our program, a music player. Thus let's start the segmentation of the codes provided above.

In the first line of **codes (1 - 18)** of the C++ file, These are the libraries used for the activity to fully functionalize and redirect the directory of the header file inside the main.cpp class. And in this line of code is also where our basic UI structure is inside the class. To be exact, our menu. Before we proceed entirely to the main program's functionality, let's state all the needs to achieve our intended outcome. As stated above, it was mentioned what are the function implementations that are needed in this activity. Thus, let's create our essentials. In order to create these supports, we need these functions to:

- **Function 1:** Create multiple nodes to be able to **store multiple songs**, and **interconnect the nodes** to one another to be able to call it a play**"LIST"**.
- **Function 2:** Create a function that takes a value parameter in order to insert it into the playlist.
- **Function 3:** Create a function that takes a specific value parameter of the node value in order to delete that specific music.
- **Function 4:** To initialize a while loop into the program to play songs in a loop and print all of the music in an organized way it was positioned.

Now that we have our essential focus, we start by introducing you on how I solved the supplementary activity. Starting in the CPP file. In the main function, **line 23 - 31**, are the initialization for the tracking of what the program is playing, creates an empty playlist to start from, a value parameter that will be taken for the song input and output for some of the functions, and the choice for the user to choose between case option on what function does he wants to call.

The whole program only consists of **while loop && switch-case function (line code: 33 - 115)**. The while loop is the overall function caller of the header file. It is the function that decides whether the user chose to call the function case 1, which is the transitioning of the current song to the next song. This accesses the list and replaces the **currentPlaying** to the next node. Represented as **currentPlaying->next**.

If you are curious too, the **line code 46 - 51**, is only for Input error checking where if a user types a letter instead of number, these lines catch the error, clean the broken input, and prevent the program from crashing.

Now let's delve more into the switch case inside the whole loop. As mentioned as before it is the loop that handles that structured switch case in a way that the program will not end yet.

The **Case 1: (line code: 57 - 53)**, as mentioned previously, is the next song function. Where this **transitions** the **current playing song to another song that was added in the list**.

The **Case 2: (line code: 65 - 71)**, is just the **opposite function of Case 1**. It transitions the current playing song to another song that was just played before the current playing song.

The **Case 3:(line code: 73 - 80),** this is the insertion of the song. Where this function creates a new object node in the header file, and adds its node inside the list.(**Hfile line code: 57 - 75**), In terms of the header file, the insertion process is just the same procedure mentioned in the singly linked list. It adds the **newNode**, which is the new object, into the parameter, which is the first node in the list. And the ***tail** initialization represents that it will be the **prev address of the *head**, so that when **newNode->prev** is initiated to the **tail node**, it will be the initialized to the prev of head (***head->prev)** since the tail was initialized to it. Stated as:

```
DoublyList<T> *tail = (*head)->prev;


    newNode->next = *head;
    newNode->prev = tail;
```

The **Case 4: (line code: 84 - 100),** is the remove function, or remove song. In terms of process, in the header file l**ine code: 92 - 111**, this ensure the process of creating a node that scrolls through the list of nodes, once the nodes finally meets the parameter it needs to delete, it deletes that current node, and fixes the flow of the list to one another again to keep the list intact. In the second case of the if structure statement inside of it, is if the intended head is what we want to delete, we directly delete equate the ***head** to **delNode,** at the same time initializing it to **delNode->Next** so that the node next to eat will inherit the head address. To be exact, it will become the new head.

The **Case 5: (line code: 102 105),** is just the basic **traversal function** where you scroll through the list and print each of the data inside those certain nodes. In terms of the **header file (line code: 33 - 54)**, Inside the function, consists of a **do while loop.** The do while loop constructs the iteration of the list. It is just a basic traversal function but with modification such as;

std::this_thread::sleep_for(std::chrono::milliseconds(1000));

This modification focuses only on the printing delay structure of the entire traversal list. So that it will be more realistic to provide a more music player approach.

Lastly, The **Case 6: (line code: 107 - 108),** is just the basic close ending program. This option is when the user feels accomplished, or is done using the program, this case executes closing the program.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google's Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used AI specially in the reversal traversal of deleting nodes, yet my own code interpretation and creation was my own structural understanding. And concepts for the supplementary for the modification of codes used from doubly linked lists, and additional libraries used to be more interactive.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.